



تشفير

T A S H F E E R

دليل المذاكرة والشرح – كيف تفهم الكود وتشرحه

الطالب: **حسين حامد الكوافي**

الرقم الدراسي: **642**

المقرن: **Cybersecurity – Encryption Algorithms**

نظرة عامة وكيف تستخدم هذا الدليل 0

هذا المستند مُعدّ لك أنت لتذاكر منه قبل العرض. لكل خوارزمية: شرح مبسّط للفكرة، ثم الكود مع تفسير كل سطر، ثم «ماذا تقول للحضور»، ثم سؤال متوقع وإجابته. احفظ الأفكار لا الكلمات – لو فهمت **لماذا** كل سطر موجود، تقدر تشرحه بثقة.

بنية المشروع باختصار

التطبيق يتكوّن من ملف خادم واحد `app.py` مبني بـ Flask يقوم بكل عمليات التشفير، وواجهة واحدة `templates/index.html` (HTML+CSS+JS). المستخدم يفتح `localhost:5000`، يختار خوارزمية، يُدخل نصاً أو ملفاً، يُدخل كلمة مرور (أو مفاتيح RSA)، فيحصل على ناتج مشفّر يقدر يفكّه لاحقاً.

مسار البيانات داخل التطبيق data flow

أو ملف → المتصفح Base64 → `app.py` دالة التشفير في → `POST /encrypt` → المتصفح (واجهة)

الواجهة لا تُشفّر شيئاً؛ كل التشفير يحدث في الخادم بلغة Python باستخدام مكتبة cryptography فقط.

جملة افتتاحية تقولها للدكتور

«بنيت أداة اسمها تشفير تدعم أربع خوارزميات تشفير حقيقية: AES و RSA و ChaCha20 و Triple-DES. مع واجهة ويب عربية. سأشرح فكرة كل خوارزمية ثم الكود الذي ينقّذها.»

1 مفاهيم لازم تتقنها قبل الشرح

لو فهمت هذه الكلمات الست، تقدر تشرح أي خوارزمية. كلها تتكرر في الكود.

متماثل مقابل غير متماثل symmetric vs asymmetric

- **متماثل (Symmetric):** مفتاح واحد يُستخدم للتشفير وفك التشفير. سريع. مثل: AES, ChaCha20, 3DES.
- **غير متماثل (Asymmetric):** مفتاحان مرتبطان – عام (يُشارَك، يُشفّر) وخاص (سرّي، يفكّ). أبطأ. مثل: RSA.

اشتقاق المفتاح PBKDF2

الفكرة ببساطة

كلمة المرور التي يكتبها المستخدم (مثل p@ss) ضعيفة وقصيرة. **PBKDF2** يحولها إلى مفتاح قوي بطول ثابت عبر تكرار دالة التجزئة SHA-256 مئة ألف مرة. التكرار الكثيف يجعل تخمين كلمة المرور بطيئاً جداً على المهاجم.

الملح وال Nonce ووسم التحقق salt · nonce/iv · tag

- **Salt (ملح):** 16 بايت عشوائية تدخل في اشتقاق المفتاح. تضمن أن نفس كلمة المرور تُنتج مفتاحاً مختلفاً كل مرة، وتُبطل جداول القرصنة الجاهزة (rainbow tables).
- **Nonce / IV:** قيمة عشوائية تُستخدم مرة واحدة لكل عملية تشفير. تضمن أن نفس النص لا يُنتج نفس الشيفرة مرتين. (في 3DES نسقيها IV).
- **Tag (وسم تحقق):** بصمة تُنتجها أوضاع GCM و Poly1305 تكشف أي تلاعب بالشيفرة؛ لو تغيّر بايت واحد يفشل فك التشفير.

Base64 والتشفير الهجين

- **Base64:** ليست تشفيراً، بل تحويل بايتات إلى نص قابل للنسخ واللصق (أحرف وأرقام). نستخدمها لإظهار الناتج المشفّر كنص.
- **Hybrid (هجين):** RSA لا يشفّر بيانات كبيرة، فنولّد مفتاح AES عشوائياً، نشفّر البيانات بـ AES، ونشفّر مفتاح AES فقط بـ RSA. وهكذا يعمل RSA مع أي حجم.

سؤال ذكي قد يسأله الدكتور

«لماذا الملح وال nonce؟» جوابك: «لمنع التكرار – بدونهما، تشفير نفس الرسالة بنفس المفتاح يُنتج دائماً نفس الشيفرة. وهذا تسريب معلومات يستغلّه المهاجم.»

symmetric · authenticated AES-256-GCM 2

الفكرة

أقوى وأشهر تشفير متماثل. «256» يعني طول المفتاح بالبت. وضع «GCM» مميّز لأنه يجمع بين **السرية** (إخفاء المحتوى) و**السلامة** (كشف التلاعب) في خطوة واحدة عبر وسم التحقق.

الكود the code

AES-256-GCM · encrypt

```
salt = os.urandom(16)
key = PBKDF2HMAC(SHA256(), 32, salt, 100_000).derive(password)
nonce = os.urandom(12)
ct = AESGCM(key).encrypt(nonce, data, None)
return base64.b64encode(salt + nonce + ct)
```

شرح كل سطر line by line

#	ماذا يفعل
1	نولّد ملحاً عشوائياً 16 بايت لهذه العملية تحديداً.
2	نشق مفتاحاً طوله 32 بايت (256 بت) من كلمة المرور عبر PBKDF2 بـ 100,000 تكرار باستخدام الملح.
3	نولّد nonce عشوائياً 12 بايت – يُستخدم مرة واحدة فقط مع هذا المفتاح.
4	AESGCM يشقّ البيانات وفي نفس الخطوة يُلحق وسم التحقق (tag) بالشفرة تلقائياً.
5	نجمع الأجزاء (ملح + nonce + شيفرة) ونحوّلها Base64 لتصبح نصاً واحداً قابلاً للنقل.

عند فك التشفير

نعكس العملية: نفصل الأجزاء بنفس الأطوال، نعيد اشتقاق المفتاح بنفس الملح، ثم
 AESGCM(key).decrypt(nonce, ct, None) يتحقق من الوسم ويُعيد النص الأصلي.

ماذا تقول للحضور

«AES هو المعيار الذهبي. ألاحظ أنني لا أحرّز المفتاح؛ أشتقه من كلمة المرور وقت الحاجة. ولأن الملح وال nonce عشوائيان، تشفير نفس النص مرتين يُعطي نتيجتين مختلفتين – وهذا مطلوب أمنياً.»

أسئلة متوقعة likely questions

س: لماذا 100,000 تكرار؟

ج: لإبطاء أي محاولة تخمين لكلمة المرور: كل تكرار يضيف تكلفة حسابية على المهاجم لكنه غير ملحوظ للمستخدم.

س: ماذا لو أدخل المستخدم كلمة مرور خاطئة عند الفك؟

ج: يفشل التحقق من ال tag فترمي المكتبة استثناء InvalidTag، ونحن نلتقطه ونُرجع رسالة خطأ واضحة بدل أن ينهار البرنامج.

س: كيف يعرف برنامج الفك أين ينتهي الملح ويبدأ ال nonce؟

ج: بالأطوال الثابتة: أول 16 بايت ملح، التالية 12 nonce، والباقي شيفرة.

asymmetric · two keys RSA-2048

3

الفكرة

تشفير بمفتاحين: المفتاح العام يُشارك مع الجميع ويُستخدم للتشفير فقط، والمفتاح الخاص سري ويُستخدم لفك التشفير. يحلّ مشكلة «كيف أرسل لك سرّاً دون أن نتفق على كلمة مرور مسبقاً».

الكود the code

RSA-2048 + Hybrid · encrypt

```
priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
pub = priv.public_key()
# Hybrid: RSA بـ AES ونشفر مفتاح AES ونشفر البيانات بـ AES
aes_key = os.urandom(32)
ct = AESGCM(aes_key).encrypt(nonce, data, None)
enc_key = pub.encrypt(aes_key, OAEP(MGF1(SHA256()), SHA256()))
return base64.b64encode(len(enc_key) + enc_key + nonce + ct)
```

شرح كل سطر line by line

#	ماذا يفعل
1	نولّد المفتاح الخاص (2048 بت) بأسّ عام قياسي 65537.
2	نشقّ المفتاح العام من الخاص – العام يُشتق من الخاص وليس العكس.
3	التشفير الهجين: نولّد مفتاح AES عشوائياً 32 بايت لتشفير البيانات الفعلية.
4	نشقّر البيانات بـ AES-GCM السريع (لأن RSA بطيء ومحدود الحجم).
5	نشقّر مفتاح AES الصغير فقط باستخدام RSA-OAEP (حشو آمن مبني على SHA-256).
6	نجمع: [طول المفتاح المشقّر] + المفتاح المشقّر + nonce + الشيفرة. ثم Base64.

ماذا تقول للحضور

«RSA يستخدم مفتاحين. لأنه بطيء ولا يشقّر إلا بيانات صغيرة، طبّقت التشفير الهجين: البيانات تُشفّر بـ AES، ومفتاح AES وحده يُشفّر بـ RSA. هكذا يعمل النظام مع أي حجم نص أو ملف.»

أسئلة متوقعة likely questions

س: لماذا لم تشقّر الملف مباشرة بـ RSA؟

ج: لأن RSA-2048 لا يشقّر إلا ~190 بايت كحد أقصى وهو بطيء جداً؛ الحل القياسي عالمياً هو التشفير الهجين.

س: ما OAEP؟

ج: نظام حشو (padding) آمن يضيف عشوائية قبل تشفير RSA، فيمنع هجمات معروفة على RSA الخام.

س: أيهما تنشر وأيهما تخفي؟

ج: تنشر العام (Public) ليشفّر لك الناس، وتخفي الخاص (Private) لأنه وحده يفكّ. مشاركة الخاص تعني انكشاف كل شيء.

stream cipher · fast ChaCha20-Poly1305 4

الفكرة

تشفير انسيابي حديث سريع جداً بالبرمجيات، مع Poly1305 للتحقق من السلامة. تختاره تطبيقات الجوال وبروتوكول TLS عندما لا يوجد تسريع عتادي لـ AES في المعالج.

الكود the code

ChaCha20-Poly1305 · encrypt

```
salt = os.urandom(16)
key = PBKDF2HMAC(SHA256(), 32, salt, 100_000).derive(password)
nonce = os.urandom(12)
ct = ChaCha20Poly1305(key).encrypt(nonce, data, None)
return base64.b64encode(salt + nonce + ct)
```

شرح كل سطر line by line

#	ماذا يفعل
1	ملح عشوائي 16 بايت كالمعتاد لاشتقاق المفتاح.
2	نشقق مفتاح 32 بايت من كلمة المرور بـ PBKDF2 (نفس نمط AES تماماً).
3	nonce عشوائي 12 بايت يُستخدم مرة واحدة.
4	ChaCha20Poly1305 يشقّر ويلحق وسم Poly1305 للتحقق – تماماً مثل GCM لكن بخوارزمية مختلفة.
5	نجمع الأجزاء ونحوّلها Base64.

ماذا تقول للحضور

«لاحظوا أن الكود شبه مطابق لـ AES – نفس فكرة الملح وال nonce والوسم. الفرق أن ChaCha20 أسرع من AES على المعالجات التي لا تملك تسريعاً عتادياً، ولهذا يُستخدم في الجوّالات.»

أسئلة متوقعة likely questions

س: ما الفرق الجوهرى عن AES؟

ج: AES تشفير كتلي (blocks) وغالباً مُسرّع عتادياً، بينما ChaCha20 انسيابي (stream) ومُحسّن للبرمجيات: كلاهما آمن وموصى به.

س: هل أقل أماناً لأنه أسرع؟

ج: لا، السرعة من تصميمه الرياضي لا من إضعاف الأمان: تعتمد Google و Cloudflare في الإنتاج.

5 legacy · educational Triple-DES (3DES)

الفكرة

يطبق خوارزمية DES القديمة ثلاث مرات بمفتاح 24 بايت لزيادة قوتها. موجود في المشروع **للتعليم فقط** ولفهم تطوّر التشفير – وليس للاستخدام الحقيقي.

الكود the code

Triple-DES (CBC) · encrypt

```
salt = os.urandom(16)
key = PBKDF2HMAC(SHA256(), 24, salt, 100_000).derive(password) # 24 bytes
iv = os.urandom(8)
padded = PKCS7(64).padder().update(data) + finalize()
ct = Cipher(TripleDES(key), CBC(iv)).encryptor().update(padded)
return base64.b64encode(salt + iv + ct)
```

شرح كل سطر line by line

#	ماذا يفعل
1	ملح عشوائي 16 بايت.
2	نشقق مفتاحاً طوله 24 بايت بالضبط (هذا ما يتطلبه 3DES) عبر PBKDF2.
3	IV عشوائي 8 بايت (حجم كتلة 3DES هو 64 بت = 8 بايت).
4	نضيف حشو PKCS7 ليصبح طول البيانات مضاعفاً لحجم الكتلة (شرط أوضاع الكتل مثل CBC).
5	نشقر بوضع CBC حيث تُربط كل كتلة بالتي قبلها لإخفاء الأنماط.
6	نجمع الملح + IV + الشيفرة ونحوّلها Base64.

ماذا تقول للحضور

«أضفته لأغراض تعليمية. لاحظوا الاختلافات: المفتاح 24 بايت، وحجم الكتلة 8 بايت فيحتاج حشو PKCS7 ووضع CBC. لكنني أتبه أنه قديم وبطيء وأهمل رسمياً – الاستخدام الحقيقي يكون بـ AES.»

أسئلة متوقعة likely questions

س: لماذا يحتاج حشو (padding) و AES-GCM لا يحتاج؟

ج: لأن CBC وضع كتلي يتطلب طولاً مضاعفاً لحجم الكتلة، بينما GCM يعمل كتدقق فلا يحتاج حشو.

س: لماذا 24 بايت تحديداً؟

ج: لأن 3DES يطبق DES ثلاث مرات وكل مفتاح 8 DES بايت، $24=8 \times 3$.

س: هل آمن اليوم؟

ج: لا يُنصح به: NIST أهمله. نعرضه للمقارنة والفهم فقط.

6 كيف يربط التطبيق كل شيء

المسارات routes

المسار	الوظيفة
GET /	يعرض الواجهة (index.html)
POST /encrypt	يشفر نصاً (يُرجع Base64) أو ملفاً (يُرجع تنزيلًا)
POST /decrypt	يفكّ التشفير، نص أو ملف
POST /generate-rsa-keys	يولد زوج مفاتيح RSA ويُرجعهما

نص أم ملف؟ text vs file

- **نص:** النتيجة تُرجع كـ JSON يحوي سلسلة Base64 تظهر في الواجهة مع زر نسخ.
- **ملف:** يُحفظ الناتج باسم tashfeer. الاسم ويُرسَل كتنزيل. عند الفك تُزيل اللاحقة لاستعادة الاسم الأصلي.
- الصور والصوت تُعامل كملفات عادية، لكن الواجهة تعرض معاينة للصورة ومشغلاً للصوت قبل التشفير.

معالجة الأخطاء

كل مسار محاط بـ `try / except` ويُرجع دائماً JSON واضحاً أو ملفاً – لا يترك المستخدم أمام صفحة فارغة ولا ينهار الخادم. كلمة مرور خاطئة → رسالة خطأ مهذبة.

كيف تختم الشرح التقني

«كل الخوارزميات تتبع نفس النمط: اشتقاق مفتاح بـ PBKDF2 + قيمة عشوائية (salt/nonce) + تشفير + تجميع + Base64. لذلك الكود متناسق وسهل القراءة.»

7 سيناريو العرض الحي – خطوة بخطوة

نقذ هذه الخطوات أمام الحضور بالترتيب. كل خطوة مكتوب فيها ماذا تفعل وماذا تقول.

1. **شغل الخادم:** في الطرفية اكتب `python app.py`. قل: «التطبيق يثبت متطلباته تلقائياً أول مرة ثم يفتح المتصفح.»
2. **افتح** `localhost:5000` – اعرض الواجهة العربية وبذل اللغة لإثبات دعم العربية والإنجليزية.
3. **AES نص:** اختر AES، اكتب «رسالة سرية»، أدخل كلمة مرور، اضغط تشفير. اعرض الناتج Base64.
4. **الفك:** انتقل لتبويب فك التشفير، الصق الناتج ونفس كلمة المرور → تظهر الرسالة الأصلية. قل: «نفس المفتاح يفك ويشفر – هذا التماثل.»
5. **كلمة مرور خاطئة:** جرّب فكاً بكلمة مرور غلط → تظهر رسالة خطأ مهذبة. قل: «وسم التحقق كشف التلاعب.»
6. **RSA:** اختر RSA، اضغط «توليد مفاتيح»، اعرض المفاتيح، شفر بالعام وافك بالخاص. قل: «مفتاحان مختلفان.»
7. **صورة:** بتبويب الصور ارفع صورة (تظهر معاينة)، شفرها ونزل ملف `tashfeer`. ثم افكّه واعرض الصورة سليمة.

احتياط قبل العرض

جرّب كل خطوة مرة في غرفتك قبل العرض. جهّز كلمة مرور تتذكّرها، وافتح الطرفية والمتصفح مسبقاً لتوفير الوقت.

8 بنك الأسئلة المتوقعة من الدكتور

س: ما الفرق بين التشفير والتجزئة (hashing)؟

ج: التشفير قابل للعكس باستخدام مفتاح، أما التجزئة (مثل SHA-256) فباتجاه واحد لا يُعكس. نستخدم SHA-256 هنا داخل PBKDF2 لاشتقاق المفتاح لا للتشفير.

س: لماذا استخدمت مكتبة cryptography ولم تكتب الخوارزميات بنفسك؟

ج: القاعدة الذهبية في الأمن: لا تبتكر تشفيرك. المكتبات المُدقّقة تتجنّب أخطاء دقيقة وخطيرة في التنفيذ.

س: أيّ خوارزمية تنصح بها للاستخدام الحقيقي؟

ج: AES-256-GCM أو ChaCha20-Poly1305 للتشفير المتماثل، و RSA لتبادل المفاتيح. تجنّب 3DES.

س: أين تُخزّن كلمة المرور؟

ج: لا تُخزّن أبداً. يُشتق منها المفتاح وقت العملية فقط ثم يُنسى. لهذا لا يمكن استرجاع البيانات لو نُسيت كلمة المرور.

س: ماذا يضمن أن الملف لم يُعبَث به؟

ج: وسم التحقق (GCM/Poly1305): أي تعديل بايت واحد يجعل الفك يفشل.

9 مسرد سريع glossary

المصطلح	المعنى المختصر
AEAD	تشفير يوفر السرية + التحقق معاً (GCM, Poly1305)
PBKDF2	اشتقاق مفتاح قوي من كلمة مرور بالتكرار
Salt	قيمة عشوائية تُفرد اشتقاق المفتاح
Nonce / IV	قيمة تُستخدم مرة واحدة لكل عملية تشفير
Tag	بصمة تحقق تكشف التلاعب
OAEP	حشو آمن لتشفير RSA
Hybrid	RSA يغلف مفتاح AES لتشفير أي حجم
Base64	تمثيل البايتات كنص (ليس تشفيراً)

جملة ختامية

«شكراً لكم. الفكرة المحورية: مفتاح قوي مشتق من كلمة المرور + عشوائية لكل عملية + تحقق من السلامة = تشفير عملي وآمن. مستعدّ لأسئلتكم.»